

# LISTA PROGRESSIVA DE EXERCÍCIOS

## Chamada de função em MIPS - do básico ao nível dos slides

Objetivo: aprender escrevendo programas completos em MIPS, com *jal*, *jr \$ra*, *\$a0-\$a3*, *\$v0*, *pilha* e chamadas aninhadas.

### Como usar esta lista

- Resolva na ordem. A lista foi montada para você aprender o sistema de chamada de função aos poucos, sem pular direto para pilha.
- Em cada exercício, escreva o programa completo em MIPS: diretiva *.text*, *.globl main*, *label main* e labels das funções.
- Não use pseudoinstruções, a menos que o professor permita. Prefira *add* para copiar registrador, *addi* para constantes pequenas, *j*, *jal*, *jr*, *beq*, *bne*, *slt*, *lw*, *sw*, *sll*.
- Quando houver chamada de função, coloque os argumentos em *\$a0*, *\$a1*, *\$a2* ou *\$a3*; pegue o retorno em *\$v0*.
- Nesta lista, coloque *nop* logo após *jal* e *jr \$ra*, seguindo a ideia dos slides com atraso de desvio.
- Não use *mul*/*mult*/*div* prontos nos exercícios em que a função *mult* deve ser feita por somas.

### Folha de apoio - o que você precisa lembrar

| Tema                     | Regra prática                                                                                                                                                                                        |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Chamar função            | Coloque os argumentos em <i>\$a0-\$a3</i> , execute <i>jal nomeDaFuncao</i> , coloque <i>nop</i> , depois copie <i>\$v0</i> para a variável de destino.                                              |
| Retornar da função       | A função deve colocar o resultado em <i>\$v0</i> e voltar com <i>jr \$ra</i> , seguido de <i>nop</i> .                                                                                               |
| <i>\$ra</i>              | <i>jal</i> salva automaticamente o endereço de retorno em <i>\$ra</i> . Se uma função chamar outra função, ela deve salvar seu <i>\$ra</i> antes da nova chamada.                                    |
| <i>\$sp</i> e pilha      | A pilha cresce para baixo: reserve espaço com <i>addi \$sp, \$sp, -4</i> , <i>-8</i> , <i>-12</i> etc.; salve com <i>sw</i> ; restaure com <i>lw</i> ; desfça com <i>addi \$sp, \$sp, +tamanho</i> . |
| Argumentos usados depois | Se a função vai sobrescrever <i>\$a0</i> ou <i>\$a1</i> para chamar outra função, mas ainda precisa do valor antigo depois, salve esse argumento na pilha.                                           |
| Retorno parcial          | Se você chama uma função, recebe <i>\$v0</i> e depois chama outra função, salve o primeiro <i>\$v0</i> em <i>\$t</i> ou na pilha antes da segunda chamada.                                           |
| Registradores do main    | Use <i>\$s0-\$s7</i> para variáveis principais do programa. Use <i>\$t0-\$t9</i> para temporários.                                                                                                   |

**Padrão de pilha recomendado para treinar no MARS:** após reservar 4 bytes com *addi \$sp, \$sp, -4*, salve em *0(\$sp)*. Para 8 bytes, use *0(\$sp)* e *4(\$sp)*. O essencial para a prova é reservar, salvar antes da chamada, restaurar depois e devolver o *\$sp* ao valor inicial.

## Bloco 1 - Primeira chamada: função sem parâmetro

### Exercício 1 - Função que retorna constante

Nível: iniciante. Objetivo: entender *jal*, *\$v0* e *jr \$ra*

```
main() {
    int a;
    a = um();
}

int um() {
```

```
    return 1;
}
```

**Mapeamento obrigatório:** a -> \$s1

**O que entregar:**

- Crie main e a função um.
- Em main, chame um com jal e copie \$v0 para \$s1.
- Na função, coloque 1 em \$v0 e retorne com jr \$ra.

**Conferência esperada:** \$s1 deve terminar com 1.

**Dica de estudo:** Este é o esqueleto mínimo de chamada de função.

## Exercício 2 - Retorno e uso após a chamada

*Nível: iniciante. Objetivo: usar o valor retornado no restante do main*

```
main() {
    int a;
    a = zero();
    a = a + 5;
}

int zero() {
    return 0;
}
```

**Mapeamento obrigatório:** a -> \$s1

**O que entregar:**

- Chame zero.
- Copie \$v0 para a.
- Depois da chamada, some 5 em a.

**Conferência esperada:** \$s1 deve terminar com 5.

**Dica de estudo:** Perceba que o programa continua na linha depois do jal.

## Exercício 3 - Duas funções independentes

*Nível: iniciante. Objetivo: fazer duas chamadas separadas*

```
main() {
    int a, b, c;
    a = um();
    b = dois();
    c = a + b;
}

int um() { return 1; }
int dois() { return 2; }
```

**Mapeamento obrigatório:** a -> \$s1, b -> \$s2, c -> \$s3

**O que entregar:**

- Implemente as duas funções.
- Chame uma função por vez.
- Copie o retorno de cada uma antes de chamar a próxima.

**Conferência esperada:** \$s1 = 1, \$s2 = 2, \$s3 = 3.

**Dica de estudo:** Não deixe o retorno de um() em \$v0 se você ainda vai chamar dois(); \$v0 será sobrescrito.

# Bloco 2 - Função com um parâmetro

## Exercício 4 - soma1

*Nível: básico. Objetivo: passar um argumento em \$a0*

```
main() {
    int a, b;
    a = 1;
    b = soma1(a);
}
```

```
int soma1(int x) {
    return x + 1;
}
```

**Mapeamento obrigatório:** a -> \$s1, b -> \$s2

**O que entregar:**

- Inicialize a = 1.
- Antes de jal soma1, coloque a em \$a0.
- Na função, calcule \$v0 = \$a0 + 1.

**Conferência esperada:** \$s2 deve terminar com 2.

**Dica de estudo:** Este é o primeiro caso com argumento.

### Exercício 5 - dobro

*Nível: básico. Objetivo: usar \$a0 dentro da função*

```
main() {
    int a, b;
    a = 4;
    b = dobro(a);
}

int dobro(int x) {
    return x + x;
}
```

**Mapeamento obrigatório:** a -> \$s1, b -> \$s2

**O que entregar:**

- Passe a por \$a0.
- Na função, faça a soma de \$a0 com \$a0.
- Retorne em \$v0.

**Conferência esperada:** \$s2 deve terminar com 8.

**Dica de estudo:** Use add, não multiplicação.

### Exercício 6 - Atualizando a própria variável

*Nível: básico. Objetivo: usar retorno para substituir o valor original*

```
main() {
    int a;
    a = 7;
    a = soma1(a);
    a = soma1(a);
}

int soma1(int x) {
    return x + 1;
}
```

**Mapeamento obrigatório:** a -> \$s1

**O que entregar:**

- Faça duas chamadas consecutivas para a mesma função.
- Em cada chamada, coloque o valor atual de a em \$a0.
- Depois de cada chamada, copie \$v0 de volta para a.

**Conferência esperada:** \$s1 deve terminar com 9.

**Dica de estudo:** Esse exercício treina o ciclo: prepara argumento -> chama -> copia retorno.

## Bloco 3 - Função com dois parâmetros

### Exercício 7 - soma simples

*Nível: básico. Objetivo: passar dois argumentos em \$a0 e \$a1*

```
main() {
    int x, y, z;
```

```

    x = 1;
    y = 2;
    z = soma(x, y);
}

int soma(int r, int s) {
    return r + s;
}

```

**Mapeamento obrigatório:** x -> \$s1, y -> \$s2, z -> \$s3

**O que entregar:**

- Coloque x em \$a0 e y em \$a1 antes da chamada.
- Na função, some \$a0 e \$a1.
- Depois da chamada, copie \$v0 para z.

**Conferência esperada:** \$s3 deve terminar com 3.

**Dica de estudo:** Este é o exemplo central dos slides.

### Exercício 8 - subtração com ordem dos argumentos

*Nível: básico. Objetivo: entender que a ordem em \$a0 e \$a1 importa*

```

main() {
    int x, y, z;
    x = 10;
    y = 4;
    z = subtrai(x, y);
}

int subtrai(int r, int s) {
    return r - s;
}

```

**Mapeamento obrigatório:** x -> \$s1, y -> \$s2, z -> \$s3

**O que entregar:**

- Coloque x em \$a0 e y em \$a1.
- Na função, faça \$v0 = \$a0 - \$a1.
- Não inverta os argumentos.

**Conferência esperada:** \$s3 deve terminar com 6.

**Dica de estudo:** Se der -6, você passou os parâmetros invertidos.

### Exercício 9 - quatro chamadas sequenciais de soma

*Nível: básico para intermediário. Objetivo: repetir o padrão dos slides com várias chamadas*

```

main() {
    int x, y, z, k, m;
    x = 1;
    y = 2;
    z = soma(x, y);
    k = soma(x, x);
    m = soma(y, y);
    x = soma(k, m);
}

int soma(int r, int s) {
    return r + s;
}

```

**Mapeamento obrigatório:** x -> \$s1, y -> \$s2, z -> \$s3, k -> \$s4, m -> \$s5

**O que entregar:**

- Use uma única função soma.
- Antes de cada jal, recarregue corretamente \$a0 e \$a1.
- Depois de cada jal, salve \$v0 no registrador correto.

**Conferência esperada:** No final: z = 3, k = 2, m = 4, x = 6.

**Dica de estudo:** Este exercício é muito parecido com o que aparece nos slides.

### Exercício 10 - mistura de funções de 1 e 2 parâmetros

*Nível: intermediário. Objetivo: controlar \$v0 entre chamadas*

```
main() {
    int a, b, c;
    a = 3;
    b = soma1(a);
    c = soma(a, b);
}

int soma1(int x) { return x + 1; }
int soma(int r, int s) { return r + s; }
```

**Mapeamento obrigatório:** a -> \$s1, b -> \$s2, c -> \$s3

**O que entregar:**

- Implemente soma1 e soma.
- Chame soma1(a), salve o retorno em b.
- Depois chame soma(a,b).

**Conferência esperada:** \$s2 = 4 e \$s3 = 7.

**Dica de estudo:** Observe que uma função pode ter uma quantidade de parâmetros diferente da outra.

## Bloco 4 - Função com laço, sem chamar outra função

### Exercício 11 - maior que zero

*Nível: intermediário. Objetivo: usar if/branch dentro da função*

```
main() {
    int a, r;
    a = 5;
    r = positivo(a);
}

int positivo(int x) {
    if (x > 0) return 1;
    else return 0;
}
```

**Mapeamento obrigatório:** a -> \$s1, r -> \$s2

**O que entregar:**

- Passe a em \$a0.
- Dentro da função, use slt para testar  $0 < x$ .
- Retorne 1 ou 0 em \$v0.

**Conferência esperada:** Como a = 5, \$s2 deve terminar com 1.

**Dica de estudo:** Evite pseudoinstruções como bgtz se o professor não permitir.

### Exercício 12 - multiplicação por somas

*Nível: intermediário. Objetivo: implementar a função mult dos slides*

```
main() {
    int i, j, k;
    j = 4;
    k = 3;
    i = mult(j, k);
}

int mult(int mando, int mdor) {
    int produto;
    produto = 0;
    while (mdor > 0) {
        produto = produto + mando;
        mdor = mdor - 1;
    }
    return produto;
}
```

**Mapeamento obrigatório:** i -> \$s0, j -> \$s1, k -> \$s2; dentro da função, escolha temporários

**O que entregar:**

- Não use instrução de multiplicação.
- Use \$a0 para mando e \$a1 para mdor.

- Use um registrador temporário para produto.
- Faça o while com slt/beq ou slt/bne.

**Conferência esperada:** \$s0 deve terminar com 12.

**Dica de estudo:** Este é outro exercício explícito dos slides.

### Exercício 13 - mult com multiplicador zero

*Nível: intermediário. Objetivo: testar o caso de parada do while*

```
main() {
    int i, j, k;
    j = 8;
    k = 0;
    i = mult(j, k);
}

int mult(int mando, int mdor) {
    int produto;
    produto = 0;
    while (mdor > 0) {
        produto = produto + mando;
        mdor = mdor - 1;
    }
    return produto;
}
```

**Mapeamento obrigatório:** i -> \$s0, j -> \$s1, k -> \$s2

**O que entregar:**

- Use a mesma função mult do exercício anterior.
- Garanta que o laço não execute quando mdor = 0.

**Conferência esperada:** \$s0 deve terminar com 0.

**Dica de estudo:** Esse teste mostra se sua condição do while está correta.

### Exercício 14 - soma de 1 até n

*Nível: intermediário. Objetivo: construir uma função com acumulador e contador*

```
main() {
    int n, r;
    n = 5;
    r = somatorio(n);
}

int somatorio(int n) {
    int soma;
    soma = 0;
    while (n > 0) {
        soma = soma + n;
        n = n - 1;
    }
    return soma;
}
```

**Mapeamento obrigatório:** n -> \$s1, r -> \$s2

**O que entregar:**

- Passe n em \$a0.
- Use acumulador temporário dentro da função.
- Retorne a soma em \$v0.

**Conferência esperada:** \$s2 deve terminar com 15.

**Dica de estudo:** Apesar de não estar literalmente nos slides, usa a mesma lógica de mult por somas.

## Bloco 5 - Função chamando outra função: salvamento de \$ra

### Exercício 15 - dois chama um

*Nível: intermediário. Objetivo: entender por que salvar \$ra*

```
main() {
    int a;
```

```

    a = dois();
    a = a + 1;
}

int dois() {
    return um() + 1;
}

int um() {
    return 1;
}

```

**Mapeamento obrigatório:** a -> \$s1

**O que entregar:**

- main chama dois.
- dois chama um, então dois precisa salvar \$ra antes de jal um.
- Depois que um retorna, dois soma 1 ao retorno e devolve em \$v0.

**Conferência esperada:** dois() retorna 2; depois main soma 1; \$s1 deve terminar com 3.

**Dica de estudo:** Este é o primeiro caso em que a pilha aparece nos slides.

### Exercício 16 - três níveis de chamada

*Nível: intermediário para avançado. Objetivo: salvar \$ra em mais de uma função*

```

main() {
    int a;
    a = tres();
}

int tres() { return dois() + 1; }
int dois() { return um() + 1; }
int um()   { return 1; }

```

**Mapeamento obrigatório:** a -> \$s1

**O que entregar:**

- main chama tres.
- tres chama dois, então tres deve salvar \$ra.
- dois chama um, então dois também deve salvar \$ra.
- um não chama ninguém, então não precisa salvar \$ra.

**Conferência esperada:** \$s1 deve terminar com 3.

**Dica de estudo:** Se o programa entra em loop no retorno, provavelmente algum \$ra não foi salvo/restaurado.

### Exercício 17 - soma3 sem precisar preservar y

*Nível: intermediário para avançado. Objetivo: chamada aninhada com alteração do argumento antes do jal*

```

main() {
    int a, b;
    a = 1;
    b = soma3(a);
    a = a + b;
}

int soma3(int y) {
    return soma1(y + 1) + 1;
}

int soma1(int x) {
    return x + 1;
}

```

**Mapeamento obrigatório:** a -> \$s1, b -> \$s2

**O que entregar:**

- main passa a em \$a0 e chama soma3.
- soma3 deve salvar \$ra, pois chama soma1.
- soma3 pode alterar \$a0 para y + 1 antes de chamar soma1.
- Depois do retorno de soma1, some 1 em \$v0.

**Conferência esperada:** Com a = 1: soma3(1) = soma1(2) + 1 = 4; depois a = 1 + 4 = 5.

**Dica de estudo:** Neste caso, y não precisa ser usado depois da chamada, então não é obrigatório salvar \$a0.

**Exercício 18 - soma3 precisando preservar y***Nível: avançado. Objetivo: salvar argumento na pilha*

```

main() {
    int a, b;
    a = 1;
    b = soma3(a);
    a = a + b;
}

int soma3(int y) {
    return soma1(y + 1) + y;
}

int soma1(int x) {
    return x + 1;
}

```

**Mapeamento obrigatório:** a -> \$s1, b -> \$s2**O que entregar:**

- soma3 deve salvar \$ra.
- soma3 também deve salvar o y original, pois vai mudar \$a0 para chamar soma1(y+1) e depois ainda precisa somar y.
- Após soma1 retornar, restaure y e faça \$v0 = retorno\_de\_soma1 + y.

**Conferência esperada:** Com a = 1: soma3(1) = soma1(2) + 1 = 4; depois a = 1 + 4 = 5.**Dica de estudo:** Esse é o exemplo dos slides em que aparece salvamento de argumento.**Exercício 19 - duas chamadas dentro da mesma função***Nível: avançado. Objetivo: salvar retorno parcial antes de nova chamada*

```

main() {
    int a, r;
    a = 3;
    r = f(a);
}

int f(int x) {
    return soma1(x) + dobro(x);
}

int soma1(int x) { return x + 1; }
int dobro(int x) { return x + x; }

```

**Mapeamento obrigatório:** a -> \$s1, r -> \$s2**O que entregar:**

- f deve salvar \$ra.
- f deve preservar x, porque chamará duas funções usando x.
- Após a primeira chamada, salve o retorno parcial antes da segunda chamada sobrescrever \$v0.

**Conferência esperada:** Com a = 3: f(3) = 4 + 6 = 10; \$s2 deve terminar com 10.**Dica de estudo:** Você pode salvar o retorno parcial em \$t ou na pilha. Se quiser ser mais seguro, use a pilha.

## Bloco 6 - Padrão máximo dos slides: função aninhada com mult e argumento preservado

**Exercício 20 - SumSquare dos slides***Nível: avançado. Objetivo: salvar \$ra e \$a1 antes de chamar mult*

```

main() {
    int a, b, c;
    a = 2;
    b = 3;
    c = SumSquare(a, b);
}

int SumSquare(int x, int y) {
    return mult(x, x) + y;
}

```



```
int mult(int m, int n) {
    int prod;
    prod = 0;
    while (n > 0) {
        prod = prod + m;
        n = n - 1;
    }
    return prod;
}
```

**Mapeamento obrigatório:** a -> \$s1, b -> \$s2, c -> \$s3

**O que entregar:**

- main passa a em \$a0 e b em \$a1.
- SumSquare recebe x em \$a0 e y em \$a1.
- Antes de chamar mult(x,x), SumSquare deve salvar \$ra.
- SumSquare também deve salvar y, pois \$a1 será trocado para receber x na chamada mult(x,x).
- mult deve ser feita por somas, sem usar multiplicação pronta.

**Conferência esperada:** Com a = 2 e b = 3: c = 2\*2 + 3 = 7.

**Dica de estudo:** Este é um dos formatos mais importantes dos slides.

### Exercício 21 - mult + soma usando os mesmos argumentos

*Nível: avançado. Objetivo: preservar argumentos para mais de uma chamada*

```
main() {
    int a, b, c;
    a = 4;
    b = 3;
    c = produtoMaisSoma(a, b);
}

int produtoMaisSoma(int x, int y) {
    return mult(x, y) + soma(x, y);
}

int mult(int m, int n) { /* multiplicação por somas */ }
int soma(int r, int s) { return r + s; }
```

**Mapeamento obrigatório:** a -> \$s1, b -> \$s2, c -> \$s3

**O que entregar:**

- produtoMaisSoma deve salvar \$ra.
- Ela precisa preservar x e y para chamar mult(x,y) e depois soma(x,y).
- Salve o retorno de mult antes de chamar soma.
- Implemente mult por somas.

**Conferência esperada:** Com a = 4 e b = 3: c = 12 + 7 = 19.

**Dica de estudo:** Este exercício força você a controlar \$a0, \$a1 e \$v0 ao mesmo tempo.

### Exercício 22 - quadradoMaisDobro

*Nível: avançado. Objetivo: misturar função com laço e função simples*

```
main() {
    int n, r;
    n = 5;
    r = quadradoMaisDobro(n);
}

int quadradoMaisDobro(int x) {
    return mult(x, x) + dobro(x);
}

int mult(int m, int n) { /* multiplicação por somas */ }
int dobro(int x) { return x + x; }
```

**Mapeamento obrigatório:** n -> \$s1, r -> \$s2

**O que entregar:**

- quadradoMaisDobro deve salvar \$ra.
- Preserve x para usar novamente depois de mult(x,x).

- Salve o retorno de mult antes de chamar dobro.

**Conferência esperada:** Com  $n = 5$ :  $r = 25 + 10 = 35$ .

**Dica de estudo:** Essa questão é excelente para treinar pilha e retorno parcial.

### Exercício 23 - função com três parâmetros

Nível: avançado. Objetivo: usar \$a0, \$a1 e \$a2

```
main() {
    int a, b, c, r;
    a = 2;
    b = 3;
    c = 4;
    r = soma3valores(a, b, c);
}

int soma3valores(int x, int y, int z) {
    return x + y + z;
}
```

**Mapeamento obrigatório:** a -> \$s1, b -> \$s2, c -> \$s3, r -> \$s4

**O que entregar:**

- Passe a em \$a0, b em \$a1 e c em \$a2.
- Na função, some os três argumentos.
- Retorne em \$v0.

**Conferência esperada:** \$s4 deve terminar com 9.

**Dica de estudo:** Embora os slides foquem mais em 0, 1 e 2 parâmetros, este exercício fecha a ideia dos registradores \$a0-\$a3.

## Bloco 7 - Vetores com chamada de função

### Exercício 24 - carregar vetor, chamar função e armazenar de volta

Nível: avançado. Objetivo: combinar lw/sw com chamada de função

```
main() {
    int VetA[...];
    int i, j;
    i = 2;
    j = VetA[i];
    j = soma1(j);
    VetA[i] = j;
}

int soma1(int x) {
    int aux;
    aux = x + 1;
    return aux;
}
```

**Mapeamento obrigatório:** base de VetA -> \$s0, i -> \$s1, j -> \$s2

**O que entregar:**

- Considere que \$s0 já contém o endereço base de VetA.
- Calcule o deslocamento de i com sll por 2.
- Use lw para carregar VetA[i].
- Passe j em \$a0 e chame soma1.
- Depois, armazene o retorno em j e em VetA[i].

**Conferência esperada:** Se VetA[2] começou com 10, ao final VetA[2] deve ser 11.

**Dica de estudo:** Este é o exercício final apresentado nos slides.

### Exercício 25 - aplicar soma1 em todo o vetor

Nível: avançado. Objetivo: fazer loop no main chamando função para cada posição

```
main() {
    int VetA[5];
    int i;
    i = 0;
```

```

while (i < 5) {
    VetA[i] = soma1(VetA[i]);
    i = i + 1;
}

int soma1(int x) { return x + 1; }

```

**Mapeamento obrigatório:** base de VetA -> \$s0, i -> \$s1; temporários à sua escolha

**O que entregar:**

- Considere VetA já preenchido na memória.
- Faça o loop no main.
- A cada iteração, calcule endereço de VetA[i], carregue, chame soma1 e grave de volta.
- Cuidado para não perder o endereço calculado durante a chamada; salve em \$t ou recalcule depois.

**Conferência esperada:** Se o vetor era [1,2,3,4,5], deve virar [2,3,4,5,6].

**Dica de estudo:** Como a função soma1 não chama outra função, ela não precisa salvar \$ra.

### Exercício 26 - somar vetor usando função soma

*Nível: avançado. Objetivo: chamar função dentro de loop*

```

main() {
    int VetA[5];
    int i, total;
    i = 0;
    total = 0;
    while (i < 5) {
        total = soma(total, VetA[i]);
        i = i + 1;
    }
}

int soma(int x, int y) { return x + y; }

```

**Mapeamento obrigatório:** base de VetA -> \$s0, i -> \$s1, total -> \$s2

**O que entregar:**

- Faça o loop no main.
- Em cada iteração, passe total em \$a0 e VetA[i] em \$a1.
- Copie \$v0 de volta para total.

**Conferência esperada:** Se o vetor era [1,2,3,4,5], total deve terminar com 15.

**Dica de estudo:** Esse formato é comum em prova porque mistura vetor, loop e chamada.

### Exercício 27 - multiplicar cada elemento por k

*Nível: muito avançado. Objetivo: usar mult por somas dentro de loop*

```

main() {
    int VetA[4];
    int i, k;
    i = 0;
    k = 3;
    while (i < 4) {
        VetA[i] = mult(VetA[i], k);
        i = i + 1;
    }
}

int mult(int m, int n) { /* multiplicação por somas */ }

```

**Mapeamento obrigatório:** base de VetA -> \$s0, i -> \$s1, k -> \$s2

**O que entregar:**

- No main, calcule o endereço do elemento.
- Carregue VetA[i], passe em \$a0 e passe k em \$a1.
- Chame mult por somas.
- Grave o retorno de volta em VetA[i].
- Cuidado para preservar i, k e/ou endereço do elemento ao redor da chamada.

**Conferência esperada:** Se VetA era [2,4,6,8], deve virar [6,12,18,24].

**Dica de estudo:** Este é um dos exercícios mais completos para treinar chamada de função sem aninhamento dentro da função chamada.

## Bloco 8 - Questões no nível máximo da sequência dos slides

### Exercício 28 - vetor com SumSquare

Nível: muito avançado. Objetivo: vetor + função aninhada + pilha

```
main() {
    int VetA[4];
    int i;
    i = 0;
    while (i < 4) {
        VetA[i] = SumSquare(VetA[i], i);
        i = i + 1;
    }
}

int SumSquare(int x, int y) {
    return mult(x, x) + y;
}

int mult(int m, int n) { /* multiplicação por somas */ }
```

**Mapeamento obrigatório:** base de VetA -> \$s0, i -> \$s1

**O que entregar:**

- main faz loop no vetor.
- Em cada iteração, passa VetA[i] em \$a0 e i em \$a1.
- SumSquare deve salvar \$ra e preservar y antes de chamar mult(x,x).
- mult deve ser feita por somas.
- Depois do retorno, main grava o resultado em VetA[i].

**Conferência esperada:** Se VetA era [1,2,3,4], deve virar [1,5,11,19].

**Dica de estudo:** Este exercício combina praticamente todos os elementos cobrados nos slides.

### Exercício 29 - duas chamadas aninhadas e vetor

Nível: muito avançado. Objetivo: salvar retorno parcial, argumento e \$ra

```
main() {
    int VetA[3];
    int i;
    i = 0;
    while (i < 3) {
        VetA[i] = processa(VetA[i], i);
        i = i + 1;
    }
}

int processa(int x, int y) {
    return SumSquare(x, y) + soma1(y);
}

int SumSquare(int x, int y) { return mult(x, x) + y; }
int soma1(int x) { return x + 1; }
int mult(int m, int n) { /* multiplicação por somas */ }
```

**Mapeamento obrigatório:** base de VetA -> \$s0, i -> \$s1

**O que entregar:**

- processa chama SumSquare e soma1, então deve salvar \$ra.
- processa deve preservar y para usar em soma1(y).
- processa deve salvar o retorno de SumSquare antes de chamar soma1.
- SumSquare também salva \$ra e preserva y antes de chamar mult.
- main deve gravar o retorno no vetor.

**Conferência esperada:** Se VetA era [2,3,4], os novos valores serão: i=0 -> 5; i=1 -> 12; i=2 -> 21.

**Dica de estudo:** Essa questão é mais difícil que os exemplos diretos, mas usa apenas peças que aparecem nos slides.

**Exercício 30 - corrigir código com erro de \$ra**

Nível: muito avançado. Objetivo: identificar bug clássico de função aninhada

```
main:
    addi $s1, $zero, 0
    jal dois
    nop
    add $s1, $v0, $zero

dois:
    jal um
    nop
    addi $v0, $v0, 1
    jr $ra
    nop

um:
    addi $v0, $zero, 1
    jr $ra
    nop
```

**Mapeamento obrigatório:** a -> \$s1

**O que entregar:**

- Explique qual é o erro do código.
- Corrija a função dois usando a pilha.
- Mostre quais instruções entram antes de jal um e antes de jr \$ra.

**Conferência esperada:** O programa corrigido deve terminar com \$s1 = 2.

**Dica de estudo:** O erro é que dois chama um e perde seu endereço de retorno original em \$ra.

**Exercício 31 - inserir nops**

Nível: muito avançado. Objetivo: treinar a regra de delay slot usada nos slides

```
main:
    addi $s1, $zero, 1
    add $a0, $s1, $zero
    jal soma1
    add $s2, $v0, $zero
    j fim
soma1:
    addi $v0, $a0, 1
    jr $ra
fim:
```

**Mapeamento obrigatório:** \$s1 e \$s2 conforme o código

**O que entregar:**

- Reescreva o código inserindo nop após cada jal, j e jr \$ra.
- Ajuste a posição da instrução que copia \$v0 para \$s2 para que ela execute depois do retorno.
- Garanta que o fluxo termine corretamente em fim.

**Conferência esperada:** Após a correção, \$s2 deve receber 2.

**Dica de estudo:** Esse tipo de questão aparece quando o professor quer saber se você entende a instrução seguinte ao desvio.

**Exercício 32 - preencher a pilha de uma função**

Nível: muito avançado. Objetivo: desenhar o frame da função

```
int f(int x, int y) {
    return soma1(x + 1) + y;
}

int soma1(int z) {
    return z + 1;
}
```

**Mapeamento obrigatório:** x em \$a0, y em \$a1

**O que entregar:**

- Diga quais valores f precisa salvar na pilha antes de chamar soma1.
- Escreva o prólogo de f.
- Escreva o epílogo de f.

- Desenhe o conteúdo da pilha após os salvamentos.

**Conferência esperada:** f precisa preservar \$ra e y. Se x não for usado após a chamada, x não precisa ser salvo.

**Dica de estudo:** Este exercício treina a justificativa, não só o código.

### Exercício 33 - programa completo final

*Nível: muito avançado. Objetivo: resolver uma questão completa como prova*

```
main() {
    int a, b, c, d;
    a = 2;
    b = 3;
    c = produtoMaisSoma(a, b);
    d = SumSquare(c, b);
}

int produtoMaisSoma(int x, int y) {
    return mult(x, y) + soma(x, y);
}

int SumSquare(int x, int y) {
    return mult(x, x) + y;
}

int soma(int r, int s) { return r + s; }
int mult(int m, int n) { /* multiplicação por somas */ }
```

**Mapeamento obrigatório:** a -> \$s1, b -> \$s2, c -> \$s3, d -> \$s4

**O que entregar:**

- Implemente todas as funções.
- Use mult por somas.
- Salve \$ra nas funções que chamam outras funções.
- Preserve argumentos que ainda serão usados depois de uma chamada interna.
- Salve retornos parciais que seriam sobrescritos por outra chamada.

**Conferência esperada:** Com a = 2 e b = 3: c = 6 + 5 = 11; d = 11\*11 + 3 = 124.

**Dica de estudo:** Este é o exercício final antes de prova.

## Checklist de correção antes de me mandar seu código

- ☐ Toda função tem label próprio?
- ☐ Antes de cada jal, os argumentos foram colocados nos registradores certos?
- ☐ Depois de cada jal, existe nop?
- ☐ Depois da chamada, o retorno foi copiado de \$v0 para a variável correta?
- ☐ Toda função retorna com jr \$ra e nop?
- ☐ A função que chama outra função salvou \$ra antes do jal interno?
- ☐ O \$ra foi restaurado antes do jr \$ra?
- ☐ O \$sp foi devolvido exatamente ao valor que tinha ao entrar na função?
- ☐ Se um argumento ainda seria usado depois de uma chamada interna, ele foi salvo?
- ☐ Se um retorno parcial em \$v0 seria necessário depois de outra chamada, ele foi salvo?
- ☐ Nos exercícios de mult, você evitou usar multiplicação pronta?
- ☐ Nos exercícios com vetor, você multiplicou o índice por 4 usando sll antes de lw/sw?

## Ordem recomendada de estudo

- Dia 1: exercícios 1 a 7. Meta: entender jal, \$a0/\$a1, \$v0 e jr \$ra.
- Dia 2: exercícios 8 a 14. Meta: várias chamadas e função com laço.
- Dia 3: exercícios 15 a 19. Meta: salvar \$ra e argumento na pilha.
- Dia 4: exercícios 20 a 23. Meta: SumSquare, mult e preservação de argumentos.
- Dia 5: exercícios 24 a 29. Meta: vetor, loop e chamadas de função.
- Revisão final: exercícios 30 a 33. Meta: corrigir erros, inserir nops e resolver programa completo.

## Modelo mental para resolver qualquer questão

1. Mapeie as variáveis do main para registradores \$s.
2. Para cada chamada, escreva quais valores entram em \$a0, \$a1, \$a2 e \$a3.
3. Escreva jal funcao e nop.
4. Logo após a chamada, copie \$v0 para o destino.
5. Dentro da função, descubra se ela chama outra função. Se chamar, salve \$ra.
6. Descubra se algum argumento será necessário depois da chamada interna. Se sim, salve esse argumento.
7. Se houver duas chamadas internas, salve o primeiro retorno antes da segunda chamada.
8. Restaure tudo na ordem correta e retorne com jr \$ra.